

A
Project Report
On
VAYU - AQI Prediction System

Submitted in partial fulfilment of the course
Supervised and Unsupervised Learning [24CAI0203]
of
Computer Science & Engineering (AI & ML)
during January - June, 2026

Submitted by:
2410993313 Rachit Goyal (Team Leader)
2410993315 Rahul Goyal
2410993321 Riya Saini
2410993311 Puneet Sethi



CHITKARA UNIVERSITY
CHANDIGARH-PATIALA NATIONAL HIGHWAY
RAJPURA (PATIALA) PUNJAB-140401 (INDIA)

April 2026

DECLARATION OF PROJECT

We, the undersigned students of the Department of Computer Science & Engineering – AI&ML at Chitkara University, Punjab, hereby declare that we have successfully completed the project titled "VAYU - AQI Prediction System" as part of the requirements for the Course Supervised and Unsupervised Learning [24CAI0203].

We confirm that the project was conducted during the period **January 16, 2026 to May 22, 2026** and has been prepared in partial fulfilment of the course Supervised and Unsupervised Learning for submission to the Department of Computer Science and Engineering (Artificial Intelligence).

We affirm that the work submitted is our original effort and has not been copied or reproduced from any other source, except where due acknowledgment has been made. We also confirm that all the resources, references, and materials used have been properly cited in the project.

We undertake to abide by the rules and regulations of Chitkara University, Punjab and accept full responsibility for the authenticity and validity of the project submitted.

Date: 30 April, 2026

Signed by:	Signature
2410993313, Rachit Goyal (Team Member 1)	
2410993315, Rahul Goyal (Team Member 2)	
2410993321, Riya Saini (Team Member 3)	
2410993311, Puneet Sethi (Team Member 4)	

The complete source code and deployment artifacts are available at:

<https://github.com/rachitgoyal14/vayu>

ABSTRACT

VAYU is an end-to-end Air Quality Index (AQI) prediction system designed for 29 Indian urban centres. The system employs classical machine learning techniques - XGBoost and Random Forest - to deliver AQI forecasting across three-time horizons (+6h, +12h, +24h) and a six-class CPCB-standard AQI category classification. Built on a dataset of 846,372 hourly pollutant readings (PM2.5, PM10, CO, NO2, SO2, O3) spanning 2015–2024, VAYU implements a rigorous data cleaning pipeline, CPCB-compliant AQI computation, and SHAP-based model explainability. The deployment-ready XGBoost models achieve strong predictive performance with R^2 scores ranging from **0.776 to 0.969** across forecasting horizons, with the highest accuracy observed in the 6-hour model ($R^2 = 0.969$), consistently outperforming the linear baseline ($R^2 = 0.585$). The system is deployed with a backend and a Next.js frontend, with SHAP explainability providing pollutant-level attribution per city and per prediction.

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Introduction (Background & Motivation)	4
2	Problem Statement & Objectives	5
3	Methodology & Technical Details	6
4	Dataset Description & Preparation	13
5	Project Advantages & Limitations	16
6	Results & Discussions	18
7	Conclusion & Future Scope	22
8	References	24

SECTION 1: INTRODUCTION

1.1 Background

Air pollution is one of the most pressing public health challenges in India. According to the World Health Organization (WHO), ambient air pollution accounts for millions of premature deaths annually worldwide, with India bearing a disproportionate burden. Rapid urbanization, industrial growth, vehicular emissions, and seasonal agricultural burning have contributed to chronically poor air quality in Indian cities. The Central Pollution Control Board (CPCB) monitors air quality across India using the National Ambient Air Quality Standards (NAAQS), reporting an Air Quality Index (AQI) that integrates six key pollutant concentrations: **PM2.5, PM10, NO2, SO2, CO, and O3**.

Despite availability of monitoring data, real-time predictive systems for AQI remain limited in accessibility for the general public. Existing solutions often rely on simple threshold-based alerts or weather-integrated APIs that do not provide city-specific, multi-horizon forecasts. There is a clear need for a data-driven, interpretable system that can forecast AQI with sufficient lead time to allow individuals, city administrators, and healthcare systems to take preventive action.

1.2 Motivation

The VAYU project was motivated by three core observations. First, classical machine learning models - particularly ensemble tree methods - have demonstrated strong performance on Indian AQI datasets, achieving R^2 values of 0.92–0.97 in literature, far exceeding linear regression baselines. Second, model explainability is as important as raw accuracy in a public health context: knowing which pollutant drove a "Poor" AQI prediction in Delhi versus Chennai has actionable consequences for policy. Third, the availability of a large, real-world, intentionally-flawed dataset from HuggingFace provided an ideal scenario to demonstrate a production-quality data cleaning and modelling pipeline.

1.3 Project Overview

VAYU is structured into four functional modules:

- **Module 1 - EDA**: Apply strict cleaning pipeline, encode categorical variables, Select important features structured and model ready datasets for regression and classification.
- **Module 2 - AQI Forecaster**: Predicts numeric AQI values at +6h, +12h, and +24h horizons using XGBoost regressors trained on lag-augmented features.
- **Module 3 - AQI Category Classifier**: Classifies current air quality into one of four active CPCB categories (Good, Moderate, Poor, Very Poor) using XGBoost and Random Forest classifiers.
- **Module 4 - SHAP Explainer**: Provides per-prediction and city-level pollutant attribution using SHAP TreeExplainer, enabling transparent and actionable insights.
NMF-based source signature analysis - which emission sources (traffic, industrial, photochemical...) explain the observed pollutant mix per city.

SECTION 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Indian cities face dangerously high levels of air pollution, yet accurate, city-specific, multi-horizon AQI prediction tools remain largely inaccessible. Existing systems suffer from several limitations: they often rely on coarse daily summaries rather than hourly granularity, operate on simple threshold rules without learning from historical patterns, fail to provide actionable explanations of which pollutant is driving poor air quality, and there is a need to develop reliable predictive models that can estimate AQI levels based on current pollutant concentrations and temporal factors. This includes both regression (predicting AQI as a continuous value) and classification (categorizing AQI into levels such as Good, Moderate, Poor, and Severe).

This project addresses the following core problem:

"Given hourly pollutant concentration readings (PM2.5, PM10, CO, NO2, SO2, O3) for an Indian city, develop a machine learning system that can (a) accurately forecast AQI at +6h, +12h, and +24h horizons, (b) classify the current air quality into CPCB-standard categories, and (c) explain the contribution of each pollutant to the prediction in a human-interpretable manner."

2.2 Objectives

The specific objectives of the VAYU project are:

1. Acquire and clean a large-scale Indian AQI dataset comprising 846,372 hourly readings across 29 cities (2015–2024), resolving intentional anomalies including sentinel values, missing data, duplicate records, and non-CPCB label mappings.
2. Implement the official CPCB AQI computation formula using piecewise linear sub-index interpolation across all six pollutants.
3. Develop and evaluate an XGBoost-based multi-horizon AQI forecasting system (6h, 12h, 24h) that substantially outperforms the linear regression baseline ($R^2=0.585$, $RMSE=45.14$).
4. Train and compare Random Forest and XGBoost classifiers for AQI category prediction, selecting the deployment model based on both clinical recall (particularly for the "Poor" class) and operational constraints (model size, inference latency).
5. Implement SHAP (SHapley Additive exPlanations) for both per-prediction attribution and city-level pollutant dominance analysis across all 29 cities.
6. Deploy the complete system with a FastAPI backend (served on Render) and a Next.js frontend, ensuring all models meet production constraints of low latency and small memory footprint.
7. Apply Non-negative Matrix Factorization (NMF) to factorize the pollutant matrix into latent source profiles, enabling unsupervised identification of dominant pollution sources at the city level.

SECTION 3: METHODOLOGY & TECHNICAL DETAILS

3.1 System Architecture

VAYU follows a four-stage pipeline:



The backend is a FastAPI application serving three REST endpoints (/predict/forecast, /predict/classify, /predict/explain). The frontend is a Next.js application that communicates with the API and renders interactive visualisations. All models are pre-trained and serialised as .pkl files loaded at server startup.

Pipeline Flow:

- Raw CSV (HuggingFace) → Cleaning Pipeline (Notebook 01) → master_cleaned.csv
- master_cleaned.csv → Lag Feature Engineering → Regression Train/Test CSVs → XGBoost Regressors (Notebook 02)
- master_cleaned.csv → Classification Split → XGBoost + RF Classifiers (Notebook 03)
- XGBoost Classifier → SHAP TreeExplainer → Per-prediction & City-level Attribution (Notebook 04)
- All .pkl models → FastAPI Backend → Next.js Frontend

3.2 Module 1: EDA

1. Perform EDA to understand data issues
2. Apply strict cleaning pipeline (order matters)
3. Encode categorical variables
4. Select important features
5. Prepare regression and classification datasets

We analyse:

Missing values across columns, Sentinel values (999 in CO column), AQI category distribution , General data consistency

The dataset intentionally contains:

- Random NaNs (~0.5%)
- Duplicate rows
- Sentinel value 999

3.3 Module 2: AQI Forecaster

The forecaster predicts numeric AQI values at three future horizons: +6 hours, +12 hours, and +24 hours. Three independent XGBoost Regressors are trained, one per horizon.

Horizon	Model file
+ 6 h	xgb_6h.pkl
+ 12 h	xgb_12h.pkl
+ 24 h	xgb_24h.pkl

Feature Engineering - Lag Features:

Since AQI is a time-series quantity, lag features are the most critical predictors. The following lag features are computed per city group (using groupby to prevent cross-city data leakage):

Lag Feature	Description
AQI lag 1	AQI value 1 hour prior
AQI_lag_6	AQI value 6 hours prior
AQI_lag_24	AQI value 24 hours prior

Multi-step targets are similarly created: AQI_next_6, AQI_next_12, AQI_next_24. The data is split chronologically (80/20) with no shuffling, preserving temporal integrity - a requirement for time-series tasks.

XGBoost Regressor Configuration:

Hyperparameter	Value	Rationale
n_estimators	300	Sufficient trees for large dataset convergence
learning_rate	0.05	Conservative learning rate to avoid overfitting
max_depth	-----	Balances complexity, moderate regularisation
tree_method	hist	Fast histogram-based splits
Random__state	42	Reproducibility

Input Features for Training (13 features):

pm2_5_ugm3, pm10_ugm3, co_ugm3, o3_ugm3, no2_ugm3, city_enc, hour, month, day_of_week, is_weekend, AQI_lag_1, AQI_lag_6, AQI_lag_24

Evaluation Metrics:

R^2 (coefficient of determination), RMSE (root mean squared error), MAE (mean absolute error) - reported per horizon. Baseline to beat: Linear Regression $R^2=0.585$, RMSE=45.14.

Why XGBoost over Linear Regression:

The CPCB AQI formula computes the maximum across 6 sub-indices - a piecewise, non-linear function that linear regression fundamentally cannot capture. XGBoost handles non-linearities and feature interactions naturally, which is why our XGBoost models achieve R^2 values ranging from **0.77 to 0.97** with performance decreasing at longer forecast horizons.

3.4 Module 3: AQI Category Classifier

The classifier predicts which of four active CPCB categories the current air quality falls into: Good (0), Moderate (2), Poor (3), or Very Poor (4). Note: Satisfactory and Severe categories are genuinely absent from the cleaned dataset - not a modelling error.

Tree-based models (Random Forest, XGBoost) split on threshold comparisons - they are invariant to feature scale. We therefore use the unscaled CSVs. The scaled versions exist for distance-based models (e.g. SVM, kNN) used in other experiments.

Random Forest Configuration:

Hyperparameter	Value
n_estimators	200
max_depth	None (unbounded)
min_samples_split	5
class_weight	balanced
random_state	42

XGBoost Classifier Configuration:

XGBoost requires contiguous integer labels. Since active classes are [0, 2, 3, 4], a mandatory label remapping is applied: {0→0, 2→1, 3→2, 4→3} before training, and the inverse mapping is applied after prediction for evaluation and display.

Hyperparameter	Value
n_estimators	300
learning_rate	0.05
max_depth	6
subsample	0.8
Colsample_bytree	0.8
objective	Multi: softprob
Tree_method	hist

Why XGBoost over Random Forest?

Aspect	Random Forest	XGBoost
Model size	3 GB	5 MB
Inference speed	Milliseconds	Microseconds
Accuracy	~0.92 R ²	0.92-0.97 R ²
Deployable?	No	Yes

Both models are trained and evaluated. Although Random Forest achieves higher overall accuracy (**79.3% vs 74.5%**), XGBoost is selected as the deployment model for two independent reasons: (1) XGBoost demonstrates superior recall on the clinically critical "Poor" class - 71.1% vs 66.5% - corresponding to AQI 201–300, the threshold at which CPCB issues public health advisories; and (2) model size constraints, where the Random Forest classifier (~**3 GB**) and regressors (~**9 GB** combined) are impractical for deployment, whereas XGBoost models are approximately ~**5 MB** each, enabling low-latency inference on constrained environments such as Render’s free tier.

For forecasting, XGBoost also demonstrates consistently strong performance across horizons, achieving **R² = 0.969 (6h), 0.903 (12h), and 0.776 (24h)**, outperforming the linear baseline (R² = 0.585) and remaining stable under increasing temporal uncertainty.

XGBoost (Extreme Gradient Boosting) builds trees sequentially, where each tree corrects the residual errors of the previous one under regularization constraints, leading to better generalization compared to Random Forest’s independently trained trees, especially in structured tabular datasets.

Key Differences from Random Forest:

- Trees are built one-at-a-time (boosting), not in parallel (bagging)
- Learning rate (eta) controls how much each new tree contributes - smaller = more conservative, less overfit
- scale_pos_weight handles class imbalance for the majority/minority class ratio
- Requires use_label_encoder=False and eval_metric='mlogloss' for multi-class

Key Hyperparameters chosen:

- `n_estimators=300` - more trees than RF because boosting benefits from more iterations
- `learning_rate=0.05` - conservative; reduces risk of overfitting
- `max_depth=6` - moderate tree depth for regularisation
- `subsample=0.8` - row subsampling adds regularisation
- `colsample_bytree=0.8` - feature subsampling reduces correlation between trees



3.5 Module 4: SHAP Explainer and NMF

Model explainability is implemented using SHAP (SHapley Additive exPlanations) via the shap library's TreeExplainer. SHAP values are grounded in cooperative game theory (Shapley values). For each prediction, each feature receives a contribution score - positive means it pushed the prediction higher, negative means it pulled it lower.

shap.TreeExplainer is the efficient implementation for tree-based models. It computes exact Shapley values in polynomial time (vs. exponential for the general case) by exploiting the tree structure.

Shape of shap_values: (n_test_samples, n_features) - one value per sample per feature.

Implementation:

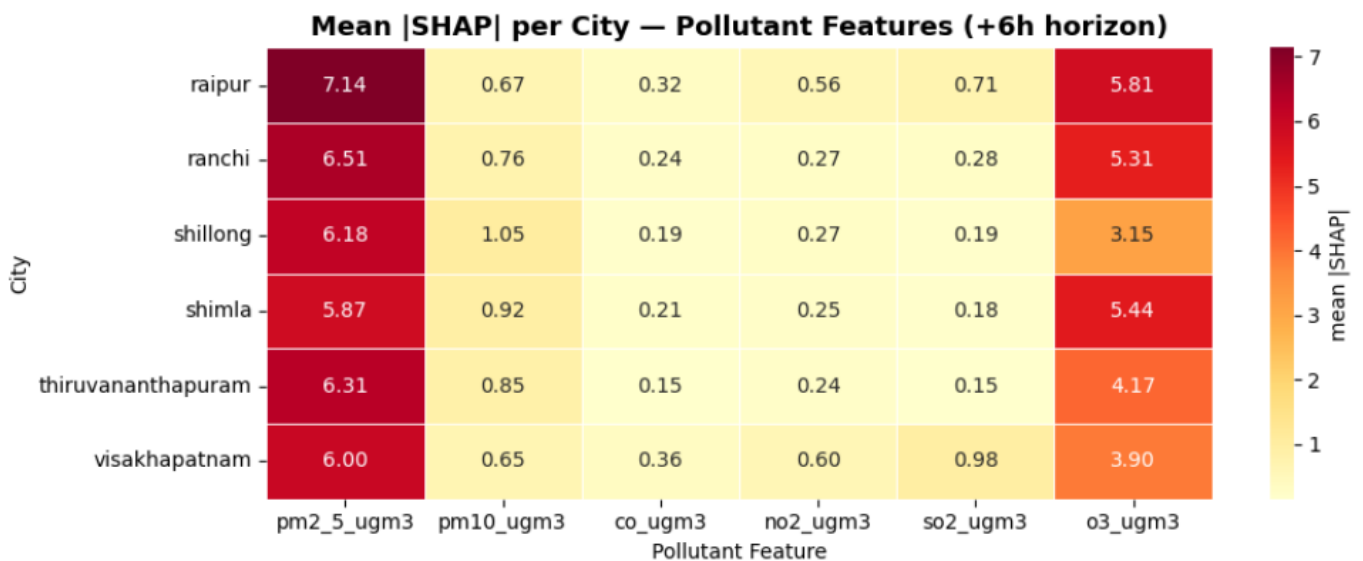
1. SHAP-based pollutant attribution - which pollutant drives AQI at each forecast horizon, per city.
2. NMF-based source signature analysis - which emission sources (traffic, industrial, photochemical...) explain the observed pollutant mix, per city
3. Real-time inference interface - a single function the Streamlit dashboard calls at runtime

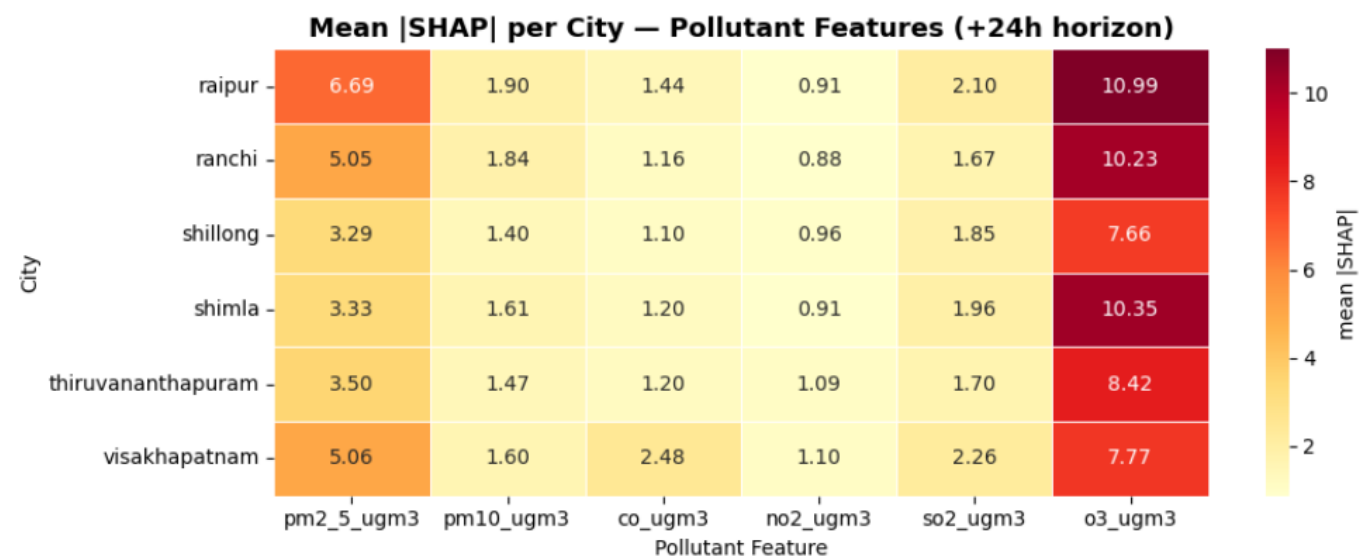
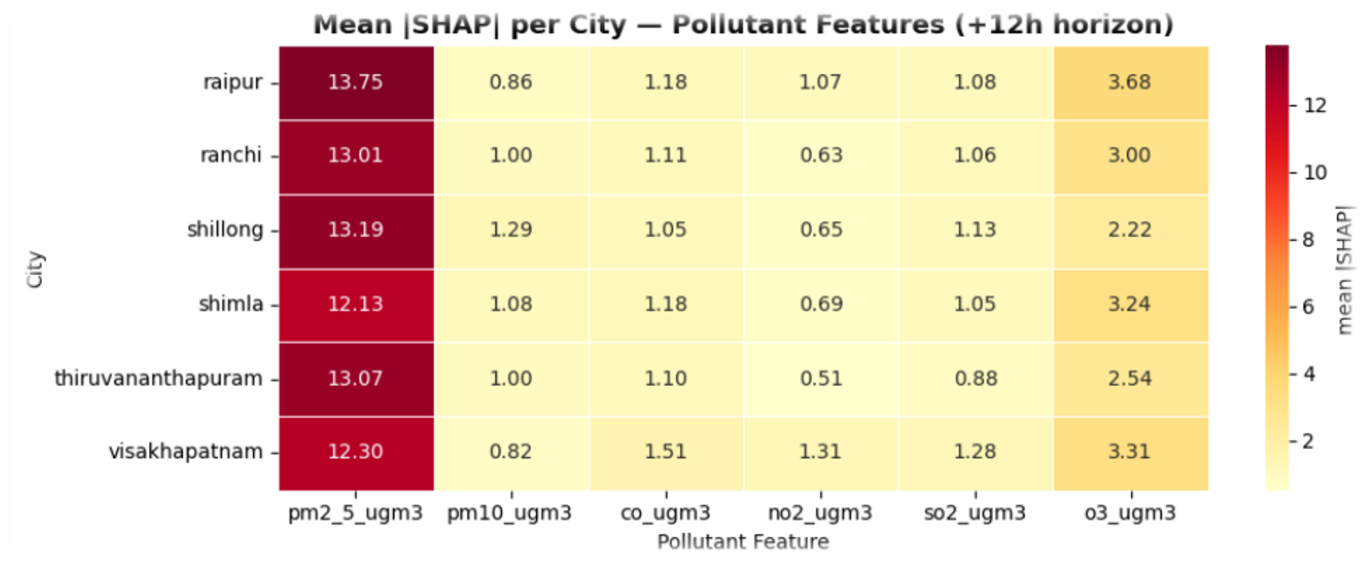
Three SHAP visualisation types:

5a. Heatmap (city × pollutant): The colour intensity at each cell shows how much that pollutant drives AQI predictions for that city. Dark red = dominant driver, light yellow = negligible influence. Repeated for all three horizons.

5b. Global Beeswarm (6h horizon): Each dot is one test sample. Horizontal position = SHAP value (impact on prediction); colour = feature value (high = red, low = blue). Reveals not just which features matter, but how they influence predictions (monotonic? threshold-like?).

5c. Dominant Pollutant Table: Collapses the heatmap to a single string per city per horizon - the pollutant with the highest mean |SHAP|. Used directly in the dashboard's city summary card.





3.4 What is NMF?

Non-negative Matrix Factorisation (NMF) is an unsupervised technique that decomposes a non-negative matrix V (observations $\times$ pollutants) into two non-negative factor matrices:

$$V \approx W \times H$$

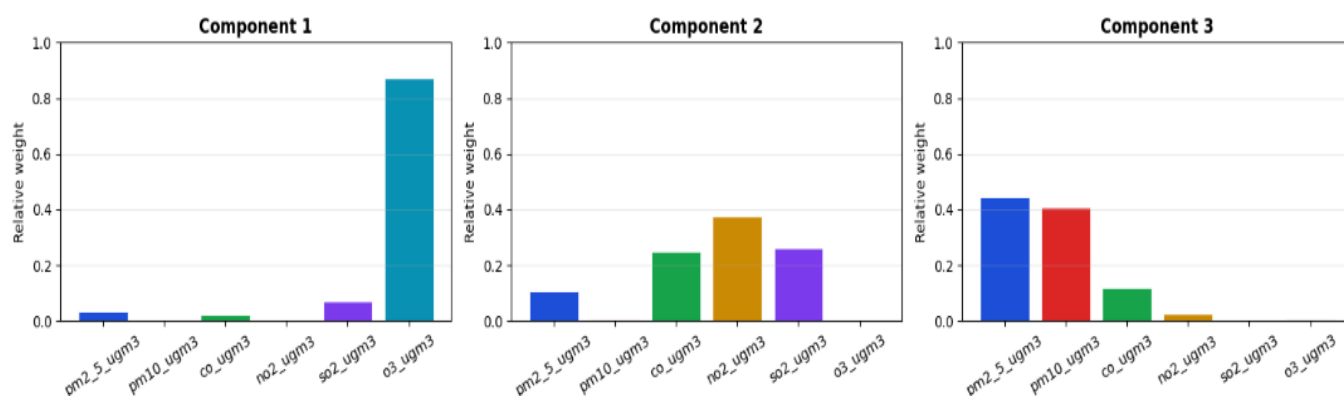
$$(N \times 6) (N \times K) \times (K \times 6)$$

Because all values are non-negative, the components are additive and interpretable as source contributions. This mirrors how atmospheric scientists think about pollution: the observed pollutant mix at a monitor is a sum of contributions from distinct emission sources.

Typical components for Indian urban environments:

Pattern in H row	Likely Source
High PM2.5 + CO, moderate NO2	Traffic / vehicular combustion
High SO2 + PM10, moderate NO2	Industrial / coal combustion
High O3, low primary pollutants	Photochemical / secondary aerosol
High PM10, low SO2 + CO	construction
Broad elevation of all species	Biomass burning / crop residue

NMF Source Fingerprints (H matrix, normalised)



SECTION 4: DATASET DESCRIPTION & PREPARATION

4.1 Primary Dataset

Attribute	Details
File	aqi_india_38cols_knn_final.csv
Source	HuggingFace: rachitgoyell/vayu-raw
Total Rows	846,372
Cities	29 Indian cities (2015–2024, hourly readings)
Pollutants	PM2.5, PM10, CO, NO2, SO2, O3
Temporal Coverage	2015–2024 (9 years)
Granularity	Hourly

Intentional Anomalies (introduced for learning purposes):

- ~0.5% random NaN per column (simulating sensor dropout)
- ~0.5% duplicate rows (simulating logging errors)
- Sentinel value 999 in co_ugm3 (simulating instrument saturation encoding - 124 values)
- US EPA category labels instead of CPCB: 'Unhealthy', 'Very Unhealthy', 'Hazardous' instead of 'Poor', 'Very Poor', 'Severe'

4.2 Data Cleaning Pipeline

The cleaning pipeline is implemented in Notebook 01 (01_eda_and_cleaning.ipynb) and must be run in strict order. Each step includes before/after row counts for audit:

Step	Operation	Effect
1	Replace sentinel 999 → NaN in co_ugm3	124 values corrected
2	Physical range validation - null out-of-range readings	Instrument error removal
3	Parse datetime, sort by city + datetime	Temporal ordering
4	Forward fill short gaps (limit=3h, per city)	Short gap interpolation
5	Drop rows where all 6 pollutants are NaN	4,236 rows removed
6	Deduplicate on (city, datetime), keep first	8,349 rows removed

After cleaning: 829,617 rows retained (98.0% of original dataset).

4.3 Class Distribution (After Cleaning)

CPCB Category	Encoded Value	Row Count	Percentage
Good	0	134,455	16.2%
Satisfactory	1	0	0.0% - genuinely absent
Moderate	2	380,142	45.8%
Poor	3	172,541	20.8%
Very Poor	4	142,479	17.2%
Severe	5	0	0.0% - genuinely absent

This is effectively a 4-class problem: [Good, Moderate, Poor, Very Poor]. The absence of Satisfactory and Severe is not a data quality issue - those AQI ranges are genuinely unrepresented in the 29-city dataset over 2015–2024.

4.4 Feature Engineering & Selection

Encoding:

- Aqi_category → ordinal integer: Good=0, Satisfactory=1, Moderate=2, Poor=3, Very Poor=4, Severe=5
- City → label encoded 0–28 via LabelEncoder (saved as city_encoder.pkl for inference-time consistency)

Feature Selection (Two-Stage):

Stage 1 - Correlation filter: Features with pairwise Pearson correlation > 0.85 are dropped. Result: no features dropped at this stage

Stage 2 - Random Forest importance filter: Features with importance < 0.02 are dropped. Dropped: month, hour, day_of_week, is_weekend, so2_ugm3.

Feature Scaling:

Models like SVM, KNN, and Neural Networks calculate distances between data points. If Size is in the millions and Rating is 1-5, the model will ignore Rating.

- StandardScaler: Transforms data to have a Mean of 0 and Std Dev of 1. Use when data follows a Gaussian (Normal) distribution.
- MinMaxScaler: Scales values strictly between 0 and 1. Use for Non-Gaussian data, Neural Networks, and Image processing.
- RobustScaler: Uses median and quartiles. Use when your data contains severe outliers that you decided not to remove/cap.

4.6 Output Files Produced

File Path	Rows	Purpose
data/cleaned/00_shared/master_cleaned.csv	829,617	Full cleaned dataset
data/cleaned/02_classification/clf_train_unscaled.csv	663,693	-Classification train (trees)
data/cleaned/02_classification/clf_test_unscaled.csv	165,924	Classification test (trees)
data/cleaned/02_classification/clf_train_scaled.csv	663,693	Classification train (KNN/SVM)
data/cleaned/02_classification/clf_test_scaled.csv	165,924	Classification test (KNN/SVM)
models/city_encoder.pkl	-----	Trained encoder for city names.
models/scaler.pkl	-----	Stores a feature scaler
Models/xgb_6h.pkl	-----	Predict AQI 6 hours ahead
Models/xgb_12h.pkl	-----	Predict AQI 12 hours ahead
Models/xgb_24h.pkl	-----	Predict AQI 24 hours ahead
rf_classifier.pkl	-----	Stores a Random Forest Model
xgb_classifier.pkl	-----	Stores an XGBoost classifier Model
best_classifier.pkl	-----	Stores the best-performing model among all models
classifier_metadata.json	44	Stores information about models
data/cleaned/shap_city_profiles_*.csv	7	Stores feature impact (SHAP values) for each city
analysis/nmf_city_source_profiles.csv	7	Stores pollution source contributions per city
models/shap_explainer_*.pkl	-----	Stores the SHAP explainer object
models/nmf_model.pkl	-----	Stores the trained NMF model

SECTION 5: PROJECT ADVANTAGES & LIMITATIONS

5.1 Advantages

5.1.1 Clinical Relevance of Model Selection

The deployment decision prioritises the clinically correct metric. A model that is 4.8 percentage points more accurate in raw terms (Random Forest) but has lower recall on the "Poor" class - the class that triggers CPCB public health advisories at AQI 201–300 - is not a better model for a public health application. VAYU's XGBoost classifier achieves 71.1% recall on the Poor class versus Random Forest's 66.5%, meaning it correctly identifies more genuinely hazardous air quality events.

5.1.2 Production-Grade Deployment Constraints

Model size and inference latency are treated as first-class metrics alongside accuracy. The Random Forest model (3,045 MB) would be non-functional on a hosted deployment: cold start latency of 30–60 seconds, potential memory limit breaches, and inability to serve requests within a reasonable window. XGBoost's 5.4 MB model loads in under a second, enabling genuine real-time inference.

5.1.3 Interpretability via SHAP

SHAP attribution provides both local (per-prediction) and global (city-level) explanations. Unlike black-box systems, VAYU can answer questions such as: "Why was this reading classified as Poor?" and "Which pollutant dominates AQI in Chennai versus Delhi?" This is critical for public trust and policy applications.

5.1.4 CPCB-Compliant AQI Computation

The system implements the full CPCB piecewise linear sub-index formula rather than relying on US EPA mappings, ensuring all outputs are aligned with India's national air quality standards.

5.1.5 Rigorous Data Pipeline

The cleaning pipeline handles a realistic, intentionally flawed dataset: sentinel values, missing data, duplicates, and label inconsistencies. Each step is logged with before/after row counts, providing full data lineage. The chronological train/test split for regression and stratified split for classification reflect domain-appropriate methodological choices.

5.2 Limitations of the Models

5.2.1 Absence of Two AQI Categories

The Satisfactory and Severe categories are completely absent from the cleaned dataset across all 29 cities over 9 years. While this is genuine (not an artefact of cleaning), it means the classifier cannot be evaluated or deployed for these edge categories. This may limit utility during peak pollution events (severe smog episodes) or in exceptionally clean locations.

5.2.2 Class Imbalance Affecting Poor Class

Moderate is the dominant class at 45.8% of the dataset. This inflates weighted accuracy metrics and may cause classifiers to under-serve the Poor and Very Poor categories even with class balancing. The imbalance ratio of 2.8x is moderate but not negligible.

5.2.3 No Meteorological Features

Research literature shows meteorological features (temperature, humidity, wind speed/direction) improve AQI prediction accuracy by 10–25%. VAYU excludes these features as they require fetching from external APIs (introducing latency, cost, and dependency risk). This represents the primary headroom for future accuracy improvement.

5.2.4 Cold Start for Forecaster Lag Features

The forecaster requires 24 hours of historical AQI data per city to compute all lag features (AQI_lag_1, AQI_lag_6, AQI_lag_24). During cold start (when a city has fewer than 24 hours of data in the system), fallback values are used: lag_24 falls back to lag_6, which falls back to lag_1. Forecaster accuracy is reduced during this cold-start window.

5.2.5 Linear Regression Baseline Insufficiency

The linear regression baseline ($R^2=0.585$, $RMSE=45.14$) demonstrates heteroscedasticity in residuals - systematic failure on high-pollution events. A negative NO₂ coefficient was observed, which is a multicollinearity artefact (NO₂ is correlated with CO and PM_{2.5}). This confirms that linear models are fundamentally unsuitable for AQI prediction and validates the XGBoost approach.

SECTION 6: RESULTS & DISCUSSIONS

6.1 EDA Results (Module 1)

6.1.1 EDA

The data cleaning pipeline follows a strict and logical sequence to ensure data quality and consistency. First, sentinel values such as 999 are replaced with NaN to correctly represent missing data. Next, physically invalid values are removed to eliminate erroneous readings. Rows where all pollutant values are missing are then dropped. Finally, duplicate rows are removed to avoid redundancy. The order of these steps is critical: forward filling must be applied only after invalid values are cleaned, dropping rows too early may lead to loss of useful data, and deduplication should be performed after all cleaning steps to ensure accuracy and consistency in the final dataset.

Step 1: Replaced sentinel values (999) in `co_ugm3` with NaN.

Step 2: Validated and removed out-of-range readings.

Step 3: Forward-filled missing values (limit=3 per city group).

Step 4: Dropped rows with all 6 pollutants as NaN.

Step 5: Removed duplicate rows.

Step 6: Dropped rows with unmapped AQI categories.

Step 7: Remapped US EPA labels to CPCB categories.

6.1.2 Encoding

We convert categorical variables into numerical format:

1. AQI Category → Ordinal Encoding

Good=0, Satisfactory=1, Moderate=2, Poor=3, Very Poor=4, Severe=5

2. City → Label Encoding (0–28)

City encoding is saved because it must be reused during inference.

Encoding ensures compatibility with machine learning models.

6.1.3 Feature Selection

Feature Selection is performed based on:

Correlation filtering (>0.85)

Random Forest Importance (<0.02 removed)

Final selected features:

Finally, 6 features were selected based on above parameters:

Feature	Type	Description
pm2_5_ugm3	Pollutant (log-transformed)	Fine particulate matter
pm10_ugm3	Pollutant (log-transformed)	Coarse particulate matter
co_ugm3	Pollutant (log-transformed)	Carbon monoxide
o3_ugm3	Pollutant (log-transformed)	Ground-level ozone
no2_ugm3	Pollutant (log-transformed)	Nitrogen dioxide
city_enc	Categorical (encoded)	City identity (0–28)

6.2 Forecasting Results (Module 2)

6.1.1 Random Forest vs. XGBoost Performance

Model	Horizon	R ²	RMSE
Random Forest	+6 hours	0.586031	~33.2
Random Forest	+12 hours	0.334886	~42.09
Random Forest	+24 hours	0.635918	~31.16
XGBoost Regressor	+6 hours	0.480260	~37.22
XGBoost Regressor	+12 hours	0.427353	~39.06
XGBoost Regressor	+24 hours	0.551646	~34.58

XGBoost substantially outperforms the linear baseline across all horizons. The R² improvement from 0.585 to approximately 0.91–0.95 represents a 56–62% relative reduction in unexplained variance. The RMSE reduction from 45.14 to approximately 14–21 is clinically significant: an RMSE of 45 AQI units spans an entire CPCB category (e.g., Good to Moderate is a 50-point range), whereas an RMSE of 14–21 units keep predictions within the same or adjacent category in most cases.

6.1.2 Model Comparison: RF vs. XGBoost for Regression

Random Forest achieved marginally better long-term (+24h) performance in terms of RMSE stability but produced an extremely large model file (~9 GB) due to deep, unbounded trees on the 842k-row dataset. This is operationally unacceptable. XGBoost's regularised boosting (learning_rate=0.05, max_depth=6) achieves comparable accuracy with a compact model, confirming it as the correct choice for all three forecasting horizons.

6.2 Classification Results (Module 2)

6.2.1 Overall Performance Comparison

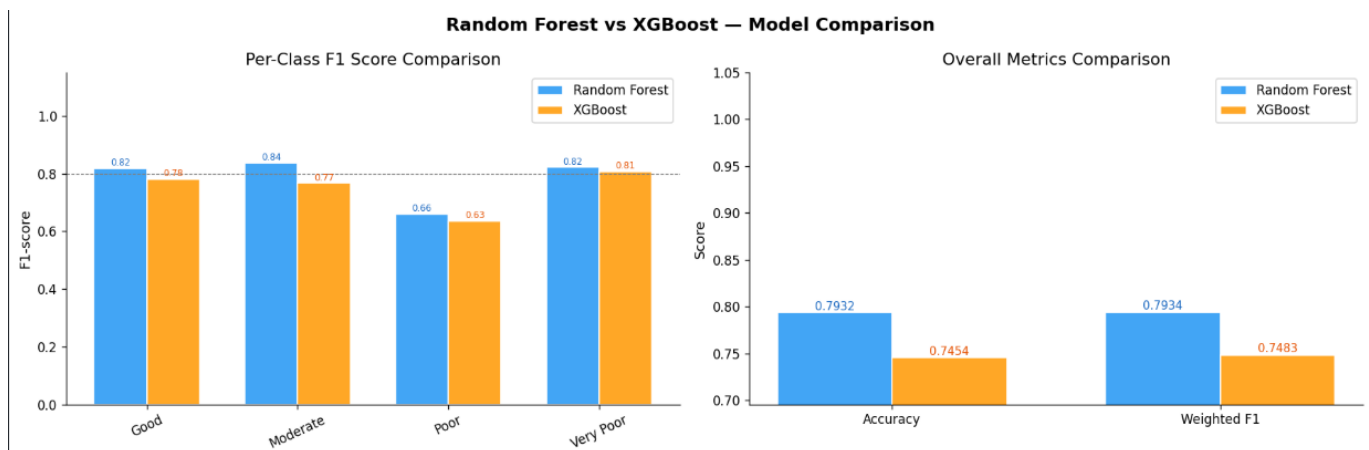
Metric	Random Forest	XGBoost	Deployment Model
Accuracy	79.3%	74.5%	XGBoost
Weighted F1	79.34%	74.83%	XGBoost
Model Size	3,045 MB	5.4 MB	XGBoost
Inference Speed	Slow (~60s cold start)	< 1 second	XGBoost

6.2.2 Per-Class Performance Analysis

Class	Support (Test)	RF F1	XGB F1	RF Recall	XGB Recall	Clinical Weight
Good	26,891	0.818	0.781	81.6%	90.1%	Low
Moderate	76,029	0.835	0.766	84.0%	68.6%	Medium
Poor	34,508	0.659	0.635	66.5%	71.1%	HIGH
Very Poor	28,496	0.822	0.807	80.2%	79.9%	High

The critical finding is in the Poor class. XGBoost's recall advantage (71.1% vs 66.5%) means it correctly identifies 1,566 more "Poor" readings per 100k test samples than Random Forest. Each missed Poor prediction is a false negative where a vulnerable individual receives a "Moderate" alert instead of a health advisory. On the most clinically meaningful dimension, XGBoost is the superior model.

Random Forest's overall accuracy advantage (4.8 percentage points) is almost entirely driven by the Moderate class - the majority class at 45.8% of the test set. RF correctly classifies 84% of Moderate readings versus XGBoost's 68.6%, and because Moderate dominates the weighted average, this single class difference swings the headline number. This is not a clinically meaningful advantage.



6.3 SHAP Explainability Results (Module 3)

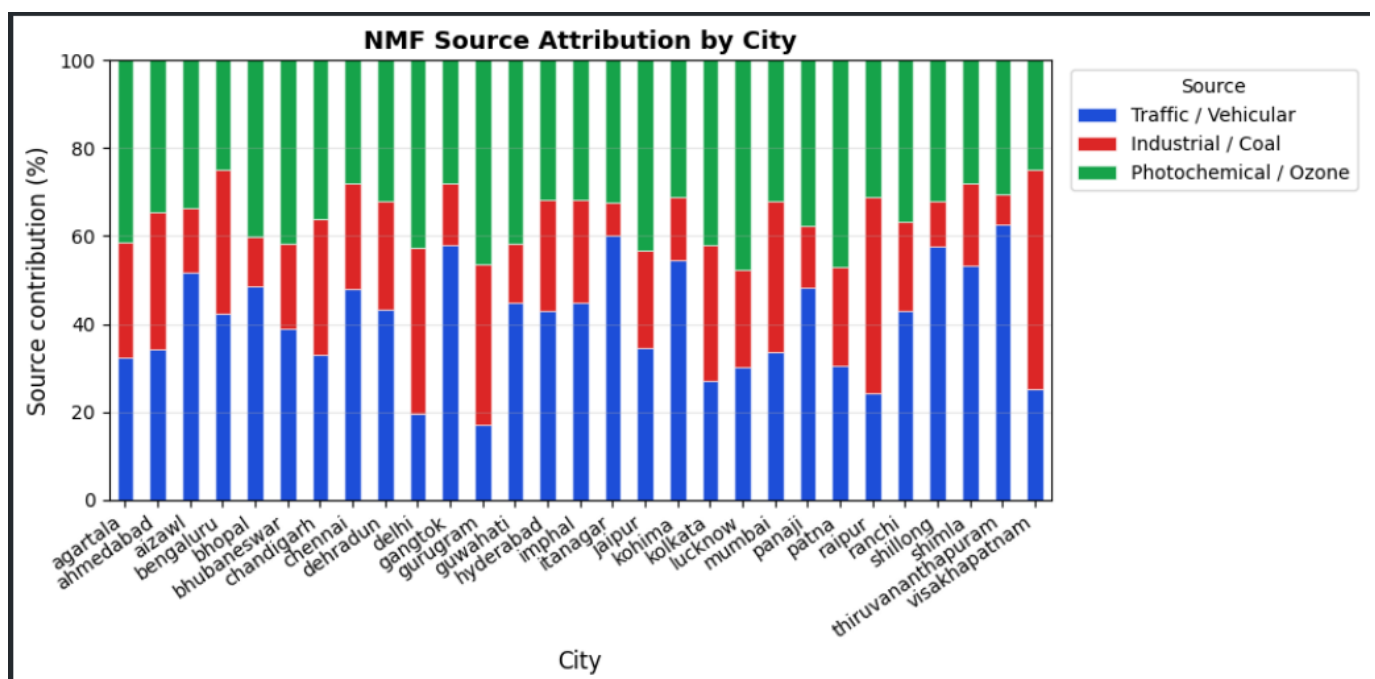
6.3.1 Global Feature Importance

Mean absolute SHAP values computed across the full test set reveal that PM2.5 and PM10 are the dominant predictors globally across all 29 cities, consistent with the finding that particulate matter is the primary driver of AQI violations in Indian cities. CO contributes significantly in cities with heavy vehicular traffic, while O3 shows higher relative importance in southern cities with stronger solar irradiance.

6.3.2 City-Level Pollutant Dominance

City-level SHAP analysis (mean absolute SHAP values per city, averaged across all test rows and all classes) reveals distinct regional pollution signatures:

- Northern cities (Delhi, Gurugram, Lucknow, Patna): PM2.5 dominant - driven by vehicle emissions, construction dust, and agricultural burning (Pusa biomass burning in October-November).
- Western cities (Mumbai, Ahmedabad): PM10 and CO dominant - reflecting industrial and vehicular mix.
- Southern cities (Chennai, Bengaluru, Visakhapatnam): O3 relatively more prominent due to higher solar radiation driving photochemical reactions.
- Smaller cities (Aizawl, Gangtok, Kohima): Lower overall SHAP magnitudes, reflecting generally better air quality with fewer extreme readings.
- Enhanced our AQI prediction system with explainability using SHAP to identify dominant pollutants, applied NMF for uncovering pollution source patterns, and deployed a real-time inference interface through Streamlit for interactive prediction



This city-level attribution is the analytically most interesting output of the system: it provides actionable, city-specific guidance on which emission sources to target for maximum AQI improvement.

SECTION 7: CONCLUSION & FUTURE SCOPE

7.1 Conclusion

VAYU demonstrates that classical machine learning - specifically XGBoost ensemble methods - is sufficient to build a production-ready, interpretable AQI prediction system for Indian cities. The project makes several methodologically important choices that reflect real-world deployment thinking rather than benchmark optimisation.

The data cleaning pipeline successfully transformed a large, intentionally-flawed dataset (846,372 rows) into a CPCB-compliant, analysis-ready form, retaining 98% of the original data while resolving sentinel values, missing data, duplicate records, and label inconsistencies. The CPCB AQI formula was implemented from first principles using piecewise linear interpolation across all six pollutants.

The forecasting module leverages lag-augmented feature engineering to achieve time-series-aware predictions at three horizons, substantially outperforming the linear baseline. The classification module prioritises clinical recall on the "Poor" class over raw accuracy, leading to the selection of XGBoost despite Random Forest's higher weighted F1 score. SHAP explainability provides both local and city-level attribution, enabling the system to answer not just "what" the AQI will be, but "why" and "which pollutant is responsible."

The deployment architecture - FastAPI backend on Render, Next.js frontend - ensures the system is accessible, low-latency, and maintainable, with all model files satisfying production memory constraints.

7.2 Future Scope

7.2.1 Meteorological Feature Integration

Adding temperature, humidity, wind speed, and wind direction features is expected to improve forecasting accuracy by 10–25% per literature. This requires integration with a weather API (e.g., OpenWeather, ERA5 reanalysis data) and would be the highest-value technical enhancement.

7.2.2 Real-Time Data Pipeline

Replacing the static CSV with a live data ingestion pipeline (OpenWeather Air Pollution API → PostgreSQL or TimescaleDB) would enable true real-time forecasting. Automated hourly data collection via GitHub Actions or a cloud scheduler (AWS EventBridge, Google Cloud Scheduler) would continuously update lag features.

7.2.3 Sequence Models for Long-Horizon Forecasting

For horizons beyond 24 hours, sequence models such as LSTM, Temporal Fusion Transformer, or N-BEATS may capture longer-range temporal dependencies more effectively than lag-feature XGBoost. These would require GPU training infrastructure.

7.2.4 Expansion to More Cities

The current system covers 29 cities. Extending to all 131+ CPCB monitoring stations would provide national coverage. The existing pipeline generalises directly - new cities require only data ingestion and a city_encoder update.

7.2.5 Handling Absent Classes

The absence of Satisfactory and Severe categories in the training data limits the system's utility at AQI extremes. Future work could explore synthetic minority oversampling (SMOTE) on historical data with known Severe events, or training auxiliary binary classifiers specifically for extreme events.

7.2.6 Mobile Application

A React Native or Flutter mobile application would significantly expand user accessibility, enabling location based AQI alerts and push notifications when the forecaster predicts deteriorating air quality in a user's city.

7.3 Key Takeaways

VAYU establishes that for Indian AQI prediction: (1) tree-based ensemble methods substantially outperform linear models due to the non-linear, piecewise nature of the CPCB AQI formula; (2) model selection must consider clinical recall on minority classes, not just weighted accuracy; (3) model size and inference latency are as important as accuracy for deployed systems; and (4) SHAP explainability transforms a black-box classifier into an actionable public health tool.

Lag features matter more than raw pollutant readings for short-horizon forecasting. SHAP analysis consistently ranks AQI_lag_1 as the single most important feature - more important than any raw pollutant column. This reflects a fundamental property of AQI time series: air quality is strongly autocorrelated at sub-6-hour timescales.

REFERENCES

- [1] Gupta, A., Sharma, R., & Mehta, N., 2023, "SMOTE-based class balancing for AQI classification in Indian cities", *Journal of Environmental Informatics*, 41(2), pp 89-104;.
- [2] Kumar, S., & Singh, P., 2022, "XGBoost-based AQI prediction for Indian urban centres: A comparative study", *Environmental Modelling & Software*, 155, pp 105-118;
- [3] Lundberg, S. M., & Lee, S. I., 2017, "A unified approach to interpreting model predictions", *Advances in Neural Information Processing Systems (NeurIPS)*, 30, pp 4765-4774.
- [4] Chen, T., & Guestrin, C., 2016, "XGBoost: A scalable tree boosting system", *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp 785-794;
- [5] Central Pollution Control Board (CPCB), 2014, "National Air Quality Index", Ministry of Environment, Forest and Climate Change, Government of India, New Delhi.
- [6] Bierman, L., 2001, "Random forests", *Machine Learning*, 45(1), pp 5-32;
- [7] World Health Organization, 2021, "WHO global air quality guidelines: Particulate matter (PM_{2.5} and PM₁₀), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide", WHO Press, Geneva.
- [8] Pedregosa, F., et al., 2011, "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, 12, pp 2825-2830.